ADVANCED SQL

FOR DATA ENGINEERING

PROFESSIONAL HANDBOOK

Database Design - Dimensional Modeling - Advanced Querying - Window Functions
Performance Tuning - Indexing - Transactions - Concurrency - Security
SCD - CDC - Partitioning - Azure Synapse - 200+ Interview Q&A - 100 Coding Problems

For: SQL Developers - Azure Data Engineers - DW Architects - DBAs - ETL Developers

PART 1: DATABASE DESIGN & DATA MODELING

Chapter 1: Entity Relationship Modeling

Learning Objectives

Understand entities, attributes, and relationships in ER modeling

Read and draw Crow's Foot notation ERDs

Model real-world domains: Retail, Banking, HR, Healthcare

Translate ER diagrams into physical SQL Server tables

1.1 What is ER Modeling?

Entity Relationship (ER) Modeling is a top-down approach to database design. It represents the data requirements of a business domain using entities (things), attributes (properties), and relationships (associations between things). The ER model is tool-independent and technology-independent -- it describes WHAT data exists, not HOW it is stored.

Concept	Definition	Example
Entity	A person, place, thing, or concept with data stored about it	Employee, Customer, Product, Order
Attribute	A property or characteristic of an entity	EmployeeID, FirstName, HireDate, Salary
Relationship	An association between two or more entities	Employee WORKS IN Department
Primary Key	Attribute(s) that uniquely identify an entity instance	EmployeeID, OrderID
Foreign Key	Attribute(s) that reference a primary key in another entity	DeptID in Employee references Department
Cardinality	Numeric relationship between entity instances	1:1, 1:N, M:N

1.2 Crow's Foot Notation

Crow's Foot notation is the most widely used ER diagram standard in industry. It uses symbols at the ends of relationship lines to show cardinality and participation.

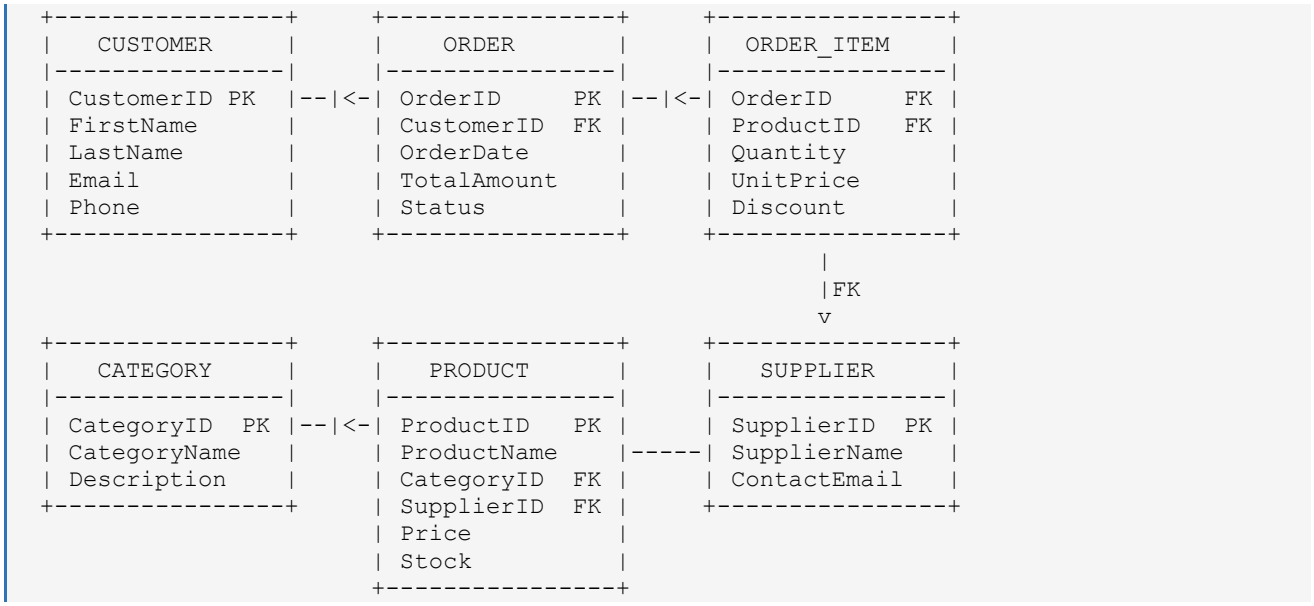
```
Crow's Foot Notation Symbols:

--|     One (exactly one)     mandatory
--O     Zero or one           optional
--<     Many                  crow's foot
--|<    One or many           mandatory many
--O<    Zero or many          optional many

Example: Customer -> Orders
```

```
CUSTOMER --|----------O<-- ORDER
           1              0..*
 One customer has zero or many orders.
 Each order belongs to exactly one customer.
```

1.3 Complete Retail ER Model



1.4 Physical SQL Implementation

```
-- Retail Model Physical Implementation
CREATE TABLE Customers (
    CustomerID   INT           IDENTITY(1,1) PRIMARY KEY,
    FirstName    NVARCHAR(50)  NOT NULL,
    LastName     NVARCHAR(50)  NOT NULL,
    Email        NVARCHAR(100) NOT NULL UNIQUE,
    Phone        VARCHAR(20),
    CreatedAt    DATETIME2     DEFAULT GETDATE()
);

CREATE TABLE Orders (
    OrderID      INT            IDENTITY(1,1) PRIMARY KEY,
    CustomerID   INT            NOT NULL REFERENCES Customers(CustomerID),
    OrderDate    DATETIME2      NOT NULL DEFAULT GETDATE(),
    Status       VARCHAR(20)    NOT NULL DEFAULT 'Pending'
        CHECK (Status IN ('Pending','Processing','Shipped','Delivered','Cancelled')),
    TotalAmount  DECIMAL(12,2)
);

CREATE TABLE OrderItems (
    OrderItemID  INT            IDENTITY(1,1) PRIMARY KEY,
    OrderID      INT            NOT NULL REFERENCES Orders(OrderID),
    ProductID    INT            NOT NULL REFERENCES Products(ProductID),
    Quantity     INT            NOT NULL CHECK (Quantity > 0),
    UnitPrice    DECIMAL(10,2)  NOT NULL,
    CONSTRAINT UQ_OrderItem UNIQUE (OrderID, ProductID)
);
```

Interview Question: What is the difference between an ERD and a physical data model?

-> An ERD shows logical entities, attributes, and relationships without implementation details. A physical data model adds SQL-specific details: data types, indexes, constraints, storage parameters. The ERD is platform-independent; the physical model is platform-specific.

Chapter 2: Database Normalization

Learning Objectives

Understand data anomalies and why normalization solves them

Apply 1NF through BCNF to unnormalized data

Recognize normal form violations in existing schemas

Know when to normalize vs when to denormalize

2.1 Why Normalization?

Normalization is the process of structuring a relational database to reduce data redundancy and improve data integrity. Unnormalized tables suffer from three types of anomalies:

Anomaly	Problem	Example
Insert Anomaly	Cannot insert partial data without unrelated data	Cannot add a new dept without an employee
Update Anomaly	Changing one fact requires updates in many rows	Manager name stored in every employee row
Delete Anomaly	Deleting a row removes unrelated important data	Deleting last employee deletes the department info

2.2 First Normal Form (1NF)

1NF requires: (1) Each column contains atomic (indivisible) values. (2) Each column contains values of a single type. (3) Each column has a unique name. (4) Order does not matter.

```
-- Violation: Projects column has multiple values
-- Fix: Move to separate row per project

CREATE TABLE Employee_Projects_1NF (
    EmpID     INT,
    EmpName   VARCHAR(50),
    DeptID    VARCHAR(10),
    DeptName  VARCHAR(50),
    MgrID     INT,
    ProjectID VARCHAR(10),
    ProjectName VARCHAR(100),
    PRIMARY KEY (EmpID, ProjectID)
);
```

2.3 Second Normal Form (2NF)

2NF requires: 1NF + No partial dependencies. Every non-key attribute must depend on the ENTIRE composite primary key, not just part of it.

```
-- Fix: Decompose into separate tables
CREATE TABLE Employees (
    EmpID    INT         PRIMARY KEY,
    EmpName  VARCHAR(50) NOT NULL,
    DeptID   VARCHAR(10) NOT NULL,
    MgrID    INT
);
```

```
CREATE TABLE Projects (
    ProjectID   VARCHAR(10)  PRIMARY KEY,
    ProjectName VARCHAR(100) NOT NULL
);

CREATE TABLE EmpProjects (
    EmpID     INT         NOT NULL REFERENCES Employees(EmpID),
    ProjectID VARCHAR(10) NOT NULL REFERENCES Projects(ProjectID),
    PRIMARY KEY (EmpID, ProjectID)
);
```

2.4 Third Normal Form (3NF)

3NF requires: 2NF + No transitive dependencies. Non-key attributes must depend only on the primary key, not on other non-key attributes.

```
-- Problem: DeptName depends on DeptID (not EmpID)
-- Fix: Separate Departments
CREATE TABLE Departments (
    DeptID   VARCHAR(10)  PRIMARY KEY,
    DeptName VARCHAR(100) NOT NULL,
    MgrID    INT
);

CREATE TABLE Employees (
    EmpID    INT         PRIMARY KEY,
    EmpName  VARCHAR(50) NOT NULL,
    DeptID   VARCHAR(10) REFERENCES Departments(DeptID)
);
```

2.5 BCNF, 4NF, 5NF

Normal Form	Rule	When to Apply
BCNF	Every determinant must be a candidate key; stricter than 3NF	When 3NF table still has functional dependency issues
4NF	No multi-valued dependencies	Rare; when one entity independently has multiple lists of values
5NF	No join dependencies not implied by candidate keys	Theoretical; very rare in practice

Practical Advice

In OLTP systems: aim for 3NF as standard baseline.

BCNF gives additional protection against anomalies -- use when designing critical financial tables.

4NF/5NF: understand for interviews; rarely applied in production beyond academic settings.

Data Warehouses are intentionally denormalized (Star Schema) for query performance.

Chapter 3: Denormalization

3.1 What is Denormalization and When to Use It

Denormalization intentionally introduces redundancy to improve read performance. It is the opposite of normalization and trades write efficiency for read speed. In OLAP/Data Warehouse environments, denormalization is the standard approach.

When to Denormalize	Reason
Reporting queries join 8+ tables	Each join is expensive at scale; flatten to wide tables
Read:Write ratio is 99:1	Redundancy cost on writes is negligible vs read gain
Hot aggregations always needed	Pre-compute summary columns rather than aggregate every query
Column store / BI tools	Wide flat tables work better with columnar engines
Response time SLA < 1 second	Normalized queries may not meet SLA under load

```
-- Denormalized flat table for reporting:
CREATE TABLE FactOrderFlat (
    OrderID       INT,
    OrderDate     DATE,
    CustomerName  NVARCHAR(100),
    CustomerEmail NVARCHAR(100),
    ProductName   NVARCHAR(200),
    CategoryName  NVARCHAR(100),
    Quantity      INT,
    UnitPrice     DECIMAL(10,2),
    LineTotal     DECIMAL(12,2)   -- pre-computed!
);
-- Single-table query -- no joins needed for reports
```

Chapter 4: Dimensional Modeling

Learning Objectives

Design Star Schema and Snowflake Schema data warehouses

Distinguish Fact tables from Dimension tables

Understand all major dimension types

Build a complete retail data warehouse schema

4.1 Star Schema

A Star Schema has a central Fact table surrounded by denormalized Dimension tables. It looks like a star. It is the standard design for analytical databases because it minimizes joins, is easy to understand, and performs well with BI tools.

```
STAR SCHEMA -- Retail Sales DW

       +--------------+
       |   DimDate    |
       +------+-------+
              |
+-----------+ |  +--------------+  +-----------+
|DimCustom |--+--|  FactSales   |--|DimProduct|
+-----------+ |  |--------------|  +-----------+
              |  | DateKey  FK  |
       +------+--| CustomerKey  |
       |DimStore | StoreKey  FK |
       +---------| ProductKey   |
                 | Quantity     |
                 | SalesAmt     |
                 +--------------+
```

4.2 Fact Table Types

Fact Type	Description	Example
Transaction Fact	One row per business transaction event	FactSales, FactOrders
Periodic Snapshot Fact	Captures state at regular time intervals	FactAccountBalance (monthly)
Accumulating Snapshot Fact	Tracks a business process from start to finish	FactOrderFulfillment

4.3 Special Dimension Types

Dimension Type	Description	Example
Conformed Dimension	Shared across multiple fact tables	DimDate, DimCustomer used by Sales and Returns
Junk Dimension	Groups low-cardinality flag/type columns	OrderFlags (IsGift, IsExpedited, IsOnline)

Dimension Type	Description	Example
Role Playing Dimension	Same dimension used multiple times in one fact	DimDate used as OrderDate, ShipDate, DeliveryDate
Degenerate Dimension	Dimension with no dimension table, stored in fact	InvoiceNumber, OrderNumber in FactSales
Slowly Changing Dimension	Tracks historical changes to dimension attributes	Customer address history -- SCD Type 2

Interview Question: What is the grain of a fact table?

-> The grain is the level of detail represented by one row in the fact table. For example, 'one row per order line item per day' is a finer grain than 'one row per order per day'. The grain must be defined before choosing what facts to include.

PART 2: ADVANCED QUERYING

Chapter 5: EXISTS and NOT EXISTS

Learning Objectives

Understand EXISTS as a semi-join and NOT EXISTS as an anti-join

Compare EXISTS vs IN vs JOIN for correctness and performance

Write correlated subqueries with EXISTS

5.1 EXISTS

EXISTS returns TRUE if the subquery returns at least one row. It is a semi-join -- it does not retrieve columns from the subquery; it only checks for row existence. EXISTS stops scanning as soon as the first match is found (short-circuits).

```
-- EXISTS: Customers who have placed at least one order
SELECT CustomerID, FirstName, Email
FROM Customers c
WHERE EXISTS (
    SELECT 1
    FROM Orders o
    WHERE o.CustomerID = c.CustomerID
);

-- NOT EXISTS: Customers with NO orders (anti-join)
SELECT CustomerID, FirstName
FROM Customers c
WHERE NOT EXISTS (
    SELECT 1 FROM Orders o
    WHERE o.CustomerID = c.CustomerID
);
```

5.2 EXISTS vs IN vs JOIN

Method	NULLs Safe	Short-circuit	Best When
EXISTS	Yes	Yes	Large correlated subquery
NOT EXISTS	Yes	Yes	Anti-join, NULLs possible
IN	No (NULL trap)	No	Small static value lists
NOT IN	No (NULL trap)	No	Only if NULLs guaranteed absent
JOIN + DISTINCT	Yes	No	Join + filter needed together

Chapter 6: ANY and ALL

```
-- ANY: returns true if ANY value in subquery satisfies condition
```

```
SELECT ProductName, Price FROM Products
WHERE Price > ANY (
    SELECT Price FROM Products WHERE Category = 'Budget'
);
-- Equivalent to: WHERE Price > MIN(Budget prices)

-- ALL: returns true only if ALL values satisfy condition
SELECT ProductName, Price FROM Products
WHERE Price > ALL (
    SELECT Price FROM Products WHERE Category = 'Budget'
);
-- Equivalent to: WHERE Price > MAX(Budget prices)
```

Chapter 7: APPLY Operators

```
-- CROSS APPLY: like INNER JOIN with a table-valued function
-- Top 3 orders per customer using CROSS APPLY
SELECT c.CustomerID, c.FirstName, top_orders.OrderID, top_orders.TotalAmount
FROM Customers c
CROSS APPLY (
    SELECT TOP 3 OrderID, TotalAmount
    FROM Orders o
    WHERE o.CustomerID = c.CustomerID
    ORDER BY TotalAmount DESC
) AS top_orders;

-- OUTER APPLY: like LEFT JOIN -- includes non-matching rows
SELECT c.CustomerID, c.FirstName, top_orders.OrderID
FROM Customers c
OUTER APPLY (
    SELECT TOP 3 OrderID, TotalAmount
    FROM Orders o
    WHERE o.CustomerID = c.CustomerID
    ORDER BY TotalAmount DESC
) AS top_orders;
```

Chapter 8: MERGE Statement

```
-- Complete MERGE: SCD Type 1 (Upsert)
MERGE DimProduct AS target
USING StagingProduct AS source
    ON target.ProductID = source.ProductID
    AND target.IsCurrent = 1

WHEN MATCHED AND (
    target.ProductName <> source.ProductName OR
    target.ListPrice   <> source.ListPrice
) THEN
    UPDATE SET
        target.ProductName = source.ProductName,
        target.ListPrice   = source.ListPrice

WHEN NOT MATCHED BY TARGET THEN
    INSERT (ProductID, ProductName, ListPrice, EffectiveDate, IsCurrent)
    VALUES (source.ProductID, source.ProductName,
            source.ListPrice, GETDATE(), 1)

WHEN NOT MATCHED BY SOURCE AND target.IsCurrent = 1 THEN
    UPDATE SET target.IsCurrent = 0, target.ExpiryDate = GETDATE();
```

MERGE Pitfalls

MERGE is not atomic by default -- add HOLDLOCK hint for concurrency safety.
Use: MERGE target WITH (HOLDLOCK) USING source ...
If source has duplicate rows on the join key, MERGE throws a runtime error.
De-duplicate source with: SELECT DISTINCT or ROW_NUMBER() before MERGE.

Chapter 9: OUTPUT Clause

```
-- OUTPUT captures inserted/deleted rows from DML
INSERT INTO Employees (Name, Salary, DeptID)
OUTPUT INSERTED.EmpID, INSERTED.Name, GETDATE() AS CreatedAt
INTO EmployeeAudit (EmpID, Name, AuditDate)
VALUES ('Alice', 85000, 3);

-- UPDATE with OUTPUT (capture before AND after)
UPDATE Employees
SET Salary = Salary * 1.10
OUTPUT DELETED.EmpID, DELETED.Salary AS OldSalary, INSERTED.Salary AS NewSalary
INTO SalaryChangeLog (EmpID, OldSalary, NewSalary)
WHERE DeptID = 3;
```

Chapter 10: TOP and OFFSET-FETCH Pagination

```
-- TOP N
SELECT TOP 10 ProductName, Price FROM Products ORDER BY Price DESC;

-- TOP WITH TIES: includes rows that tie for the last position
SELECT TOP 3 WITH TIES ProductName, Price
FROM Products ORDER BY Price DESC;

-- OFFSET-FETCH: Standard pagination (SQL Server 2012+)
DECLARE @Page INT = 3, @PageSize INT = 20;
SELECT ProductID, ProductName, Price
FROM Products
ORDER BY Price DESC
OFFSET (@Page - 1) * @PageSize ROWS
FETCH NEXT @PageSize ROWS ONLY;
```

PART 3: ADVANCED WINDOW FUNCTIONS

Chapters 11-15: Window Functions Complete Guide

Learning Objectives

Master OVER(), PARTITION BY, ORDER BY, and frame clauses
Use ROW_NUMBER, RANK, DENSE_RANK, NTILE correctly
Apply LEAD, LAG, FIRST_VALUE, LAST_VALUE for time-series analysis
Build running totals, moving averages, and rolling windows
Use PERCENT_RANK and CUME_DIST for statistical reporting

Window Function Anatomy

```
-- Complete OVER() syntax:
function_name([arguments])
OVER (
    [PARTITION BY partition_columns]
    [ORDER BY sort_columns]
    [ROWS | RANGE BETWEEN
        {UNBOUNDED PRECEDING | n PRECEDING | CURRENT ROW}
        AND
        {CURRENT ROW | n FOLLOWING | UNBOUNDED FOLLOWING}]
)

-- Frame options:
-- ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW  => cumulative
-- ROWS BETWEEN 6 PRECEDING AND CURRENT ROW          => 7-row rolling
-- ROWS BETWEEN 2 PRECEDING AND 2 FOLLOWING          => centered window
```

Ranking Functions

```
SELECT
    SalesRep,
    Month,
    Revenue,
    -- ROW_NUMBER: always unique (no ties)
    ROW_NUMBER()  OVER (PARTITION BY Month ORDER BY Revenue DESC) AS RowNum,
    -- RANK: gaps on ties (1,1,3)
    RANK()        OVER (PARTITION BY Month ORDER BY Revenue DESC) AS Rnk,
    -- DENSE_RANK: no gaps on ties (1,1,2)
    DENSE_RANK()  OVER (PARTITION BY Month ORDER BY Revenue DESC) AS DenseRnk,
    -- NTILE: divide into N roughly equal buckets
    NTILE(4)      OVER (PARTITION BY Month ORDER BY Revenue DESC) AS Quartile
FROM MonthlySales;

-- Practical: Top 3 products per category
WITH Ranked AS (
    SELECT Category, ProductName, SalesAmt,
           ROW_NUMBER() OVER (PARTITION BY Category ORDER BY SalesAmt DESC) AS rn
    FROM ProductSales
)
SELECT Category, ProductName, SalesAmt
FROM Ranked WHERE rn <= 3
ORDER BY Category, rn;
```

Analytical Functions: LEAD, LAG, FIRST_VALUE, LAST_VALUE

```
SELECT
    SaleDate, Revenue,
    -- LAG: look back N rows
    LAG(Revenue, 1, 0)  OVER (ORDER BY SaleDate) AS PrevDayRevenue,
    -- LEAD: look ahead N rows
    LEAD(Revenue, 1, 0) OVER (ORDER BY SaleDate) AS NextDayRevenue,
    -- Day-over-day change
    Revenue - LAG(Revenue,1,0) OVER (ORDER BY SaleDate) AS DayChange,
    -- FIRST_VALUE
    FIRST_VALUE(Revenue) OVER (PARTITION BY YEAR(SaleDate) ORDER BY SaleDate
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS YearStartRevenue,
    -- LAST_VALUE requires full frame
    LAST_VALUE(Revenue) OVER (PARTITION BY YEAR(SaleDate) ORDER BY SaleDate
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS YearEndRevenue
FROM DailySales ORDER BY SaleDate;
```

Running Totals, Moving Averages, Rolling Windows

```
SELECT SaleDate, Revenue,
    -- Cumulative sum (running total)
    SUM(Revenue) OVER (
        ORDER BY SaleDate
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS CumulativeRevenue,
    -- 7-day moving average
    AVG(Revenue) OVER (
        ORDER BY SaleDate
        ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
    ) AS MovAvg7Day,
    -- Year-to-date total
    SUM(Revenue) OVER (
        PARTITION BY YEAR(SaleDate)
        ORDER BY SaleDate
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS YTDRevenue
FROM DailySales ORDER BY SaleDate;
```

Statistical Functions: PERCENT_RANK, CUME_DIST

```
SELECT EmpID, Name, Salary,
    PERCENT_RANK() OVER (ORDER BY Salary) AS SalaryPercentile,
    CUME_DIST()    OVER (ORDER BY Salary) AS CumDist,
    CASE
        WHEN PERCENT_RANK() OVER (ORDER BY Salary) >= 0.9 THEN 'Top 10%'
        WHEN PERCENT_RANK() OVER (ORDER BY Salary) >= 0.75 THEN 'Top 25%'
        WHEN PERCENT_RANK() OVER (ORDER BY Salary) >= 0.5 THEN 'Above Median'
        ELSE 'Below Median'
    END AS SalaryBand
FROM Employees;

-- Median calculation
SELECT DISTINCT DeptID,
    PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY Salary)
        OVER (PARTITION BY DeptID) AS MedianSalary
FROM Employees;
```

Interview Question: What is the difference between RANK and DENSE_RANK?

-> RANK leaves gaps after ties: if two rows tie for position 2, both get rank 2 and the next row gets rank 4. DENSE_RANK does not leave gaps: ties both get rank 2 and the next row gets rank 3. Use DENSE_RANK for top-N-per-group queries.

PART 4: SQL PERFORMANCE TUNING

Chapters 16-21: Execution Plans, Indexes, Statistics, Query Store

Learning Objectives

Read and interpret SQL Server execution plans
Understand index seek vs scan decisions
Use statistics and Query Store for optimization
Diagnose and fix parameter sniffing
Analyze wait statistics for bottleneck identification

Chapter 16: Query Execution Plans

```
-- Key operators in execution plans:
-- Table Scan       => reads every row (no index or unusable index)
-- Index Seek       => uses index B-tree to jump to rows (FAST)
-- Key Lookup       => fetches non-index columns from clustered index
-- Hash Match       => join/aggregate using hash table (memory)
-- Nested Loops     => for each outer row, loop inner rows (small sets)
-- Merge Join       => requires both inputs sorted (very fast if sorted)
-- Sort             => explicit sort step (expensive -- avoid if possible)

-- WARNING signs:
-- Fat arrows       => high row counts between operators
-- Spill warnings   => spilled to tempdb (slow)
-- Key Lookup       => covering index opportunity
-- Implicit convert => data type mismatch (kills index seek!)
```

Chapter 17: Index Seek vs Scan

Operation	Description	When Used
Table Scan	Reads every data page in the table	No usable index exists
Index Seek	Uses B-tree to navigate to qualifying rows	WHERE on indexed column, sargable predicate
Key Lookup	Fetch from clustered index for missing columns	Non-covering index -- add INCLUDE to fix

```
-- NON-SARGABLE (function on column = index scan):
SELECT * FROM Employees WHERE YEAR(HireDate) = 2023;
SELECT * FROM Employees WHERE UPPER(LastName) = 'SMITH';

-- SARGABLE (index seek):
SELECT * FROM Employees
WHERE HireDate >= '2023-01-01' AND HireDate < '2024-01-01';
SELECT * FROM Employees WHERE LastName = 'Smith';

-- Implicit conversion trap (kills index seek):
SELECT * FROM Employees WHERE EmpID = '101';  -- implicit convert!
SELECT * FROM Employees WHERE EmpID = 101;    -- correct
```

Chapter 19: Query Store

```
-- Enable Query Store
ALTER DATABASE SalesDB SET QUERY_STORE = ON;

-- Find top 10 most expensive queries
SELECT TOP 10
    qt.query_sql_text,
    rs.avg_duration / 1000.0 AS AvgDuration_ms,
    rs.count_executions      AS ExecutionCount
FROM sys.query_store_query q
JOIN sys.query_store_query_text qt ON q.query_text_id = qt.query_text_id
JOIN sys.query_store_plan p        ON q.query_id = p.query_id
JOIN sys.query_store_runtime_stats rs ON p.plan_id = rs.plan_id
ORDER BY rs.avg_duration DESC;

-- Force a specific plan
EXEC sp_query_store_force_plan @query_id = 42, @plan_id = 7;
```

Chapter 20: Parameter Sniffing

Parameter sniffing occurs when SQL Server compiles a cached execution plan based on the first parameter values used. If those values are not representative, the cached plan performs poorly for other parameter values.

```
-- Solution 1: OPTION(RECOMPILE) -- recompile every execution
SELECT * FROM Orders WHERE CustomerID = @CustomerID
OPTION (RECOMPILE);

-- Solution 2: Optimize for unknown
SELECT * FROM Orders WHERE CustomerID = @CustomerID
OPTION (OPTIMIZE FOR (@CustomerID UNKNOWN));

-- Solution 3: Local variable trick
DECLARE @LocalID INT = @CustomerID;
SELECT * FROM Orders WHERE CustomerID = @LocalID;
```

Chapter 21: Wait Statistics

Wait Type	Category	Meaning	Action
CXPACKET	CPU	Parallel query coordination	Tune MAXDOP, fix underlying query
LCK_M_XX	Locking	Waiting for row/page lock	Find blocking chain, optimize transactions
PAGEIOLATCH_SH	I/O	Reading data page from disk	Add SSD, increase buffer pool, add indexes
WRITELOG	I/O	Log buffer flush to disk	Faster disk for log files
RESOURCE_SEMAPHORE	Memory	Waiting for query memory grant	Increase server memory or tune queries

PART 5: ADVANCED INDEXING

Chapters 22-29: Complete Indexing Guide

Learning Objectives

Design composite, covering, and filtered indexes

Understand Columnstore indexes for analytics workloads

Detect and fix index fragmentation

Design partitioned indexes for large DW tables

Composite, Covering, and Filtered Indexes

```
-- COMPOSITE INDEX -- Column order matters!
CREATE NONCLUSTERED INDEX IX_Orders_Cust_Date
ON Orders (CustomerID ASC, OrderDate DESC);
-- Supports: WHERE CustomerID = x AND OrderDate > y
-- Does NOT help: WHERE OrderDate > y alone

-- COVERING INDEX (with INCLUDE) -- Eliminates key lookup
CREATE NONCLUSTERED INDEX IX_Orders_Cover
ON Orders (CustomerID, OrderDate)
INCLUDE (TotalAmount, Status, ShipDate);

-- FILTERED INDEX -- Smaller, faster for a subset of rows
CREATE NONCLUSTERED INDEX IX_Orders_Active
ON Orders (CustomerID, OrderDate)
WHERE Status = 'Active';

-- UNIQUE FILTERED INDEX
CREATE UNIQUE NONCLUSTERED INDEX UX_Employees_Email
ON Employees (Email)
WHERE Email IS NOT NULL;
```

Columnstore Indexes

Columnstore indexes store data by column rather than row. They are ideal for analytical (OLAP) queries that scan large numbers of rows but only need a few columns. They can deliver 10-100x compression and 100x query performance for aggregation-heavy queries.

```
-- Clustered Columnstore Index (replaces B-tree clustered index)
CREATE CLUSTERED COLUMNSTORE INDEX CCI_FactSales ON FactSales;

-- Nonclustered Columnstore (OLAP performance on OLTP table)
CREATE NONCLUSTERED COLUMNSTORE INDEX NCCI_Orders_Analytics
ON Orders (CustomerID, OrderDate, TotalAmount, Status);

-- Azure Synapse: Clustered Columnstore is the default
CREATE TABLE FactSales (
    DateKey     INT NOT NULL,
    CustomerKey INT NOT NULL,
    SalesAmount DECIMAL(12,2)
)
WITH (
    CLUSTERED COLUMNSTORE INDEX,
    DISTRIBUTION = HASH(CustomerKey)
);
```

Index Fragmentation

```
-- Check fragmentation
SELECT
    OBJECT_NAME(ips.object_id) AS TableName,
    i.name                     AS IndexName,
    ips.avg_fragmentation_in_percent,
    ips.page_count
FROM sys.dm_db_index_physical_stats(
    DB_ID(), NULL, NULL, NULL, 'LIMITED') ips
JOIN sys.indexes i
    ON ips.object_id = i.object_id AND ips.index_id = i.index_id
WHERE ips.avg_fragmentation_in_percent > 10
ORDER BY ips.avg_fragmentation_in_percent DESC;

-- REORGANIZE: online, lightweight, < 30% fragmentation
ALTER INDEX IX_Orders_Cover ON Orders REORGANIZE;

-- REBUILD: thorough, > 30% fragmentation
ALTER INDEX IX_Orders_Cover ON Orders REBUILD;
ALTER INDEX ALL ON Orders REBUILD WITH (ONLINE = ON);
```

PART 6: TRANSACTIONS, LOCKING & CONCURRENCY

Chapters 30-34: Isolation Levels, Locks, Deadlocks

Learning Objectives

Understand all five isolation levels and their trade-offs

Identify dirty reads, non-repeatable reads, and phantom reads

Diagnose and resolve deadlocks from deadlock graphs

Monitor and unblock blocking sessions

Isolation Levels

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read	Performance
Read Uncommitted	Yes	Yes	Yes	Best -- no shared locks taken
Read Committed (default)	No	Yes	Yes	Good -- shared locks released after read
Repeatable Read	No	No	Yes	Moderate -- shared locks held to end
Serializable	No	No	No	Worst -- range locks, full serialization
Snapshot (RCSI/SI)	No	No	No	Best for reads -- uses row versioning

```
-- Set isolation level
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
SET TRANSACTION ISOLATION LEVEL SNAPSHOT;

-- Enable Snapshot isolation
ALTER DATABASE SalesDB SET ALLOW_SNAPSHOT_ISOLATION ON;
ALTER DATABASE SalesDB SET READ_COMMITTED_SNAPSHOT ON;
```

Deadlocks -- Detection and Resolution

```
-- Enable deadlock trace
DBCC TRACEON (1222, -1);

-- Deadlock prevention:
-- 1. Always access objects in the same order
-- 2. Keep transactions short
-- 3. Use SNAPSHOT isolation to eliminate reader-writer deadlocks
-- 4. Use SET DEADLOCK_PRIORITY LOW for less critical sessions
SET DEADLOCK_PRIORITY LOW;
```

Blocking -- Monitor and Resolve

```
-- Find blocking chains
SELECT
    r.session_id,
    r.blocking_session_id,
    r.wait_type,
    r.wait_time / 1000  AS WaitSec,
    r.status
FROM sys.dm_exec_requests r
WHERE r.blocking_session_id > 0
ORDER BY r.wait_time DESC;

-- Kill blocking session (use with caution)
KILL 55;
```

PART 7: ADVANCED VIEWS

Indexed Views (Materialized Views)

```
-- Requirements: WITH SCHEMABINDING, UNIQUE CLUSTERED index
CREATE VIEW dbo.vw_SalesSummary
WITH SCHEMABINDING AS
SELECT
    s.CustomerID,
    p.Category,
    COUNT_BIG(*)       AS OrderCount,
    SUM(s.SalesAmount) AS TotalSales
FROM dbo.FactSales s
JOIN dbo.DimProduct p ON s.ProductKey = p.ProductKey
GROUP BY s.CustomerID, p.Category;

CREATE UNIQUE CLUSTERED INDEX UCI_vw_SalesSummary
ON dbo.vw_SalesSummary (CustomerID, Category);
```

PART 8: SQL SECURITY

Row Level Security (RLS)

```
-- Step 1: Create filter function
CREATE FUNCTION dbo.fn_SecurityFilter(@SalesRegion VARCHAR(50))
RETURNS TABLE WITH SCHEMABINDING AS RETURN (
    SELECT 1 AS result
    WHERE @SalesRegion = USER_NAME()
       OR USER_NAME() = 'SalesManager'
       OR IS_MEMBER('db_owner') = 1
);

-- Step 2: Create security policy
CREATE SECURITY POLICY SalesRegionPolicy
ADD FILTER PREDICATE dbo.fn_SecurityFilter(SalesRegion)
ON dbo.SalesData WITH (STATE = ON);

-- Dynamic Data Masking
ALTER TABLE Employees
ALTER COLUMN Email ADD MASKED WITH (FUNCTION = 'email()');
-- Non-privileged user sees: eXXX@XXXX.com
```

PART 9: ADVANCED SQL SERVER OBJECTS

Triggers

```
-- AFTER INSERT trigger for audit log
CREATE OR ALTER TRIGGER trg_Employees_AfterInsert
ON Employees AFTER INSERT AS BEGIN
    SET NOCOUNT ON;
    INSERT INTO EmployeeAuditLog (EmpID, Action, ChangedBy, ChangedAt)
    SELECT EmpID, 'INSERT', SYSTEM_USER, GETDATE() FROM INSERTED;
END;

-- Parameterized dynamic SQL
DECLARE @stmt NVARCHAR(500) = N'SELECT * FROM Employees WHERE DeptID = @DeptID';
DECLARE @params NVARCHAR(100) = N'@DeptID INT';
EXEC sp_executesql @stmt, @params, @DeptID = 3;
```

PART 10: DATA WAREHOUSE SQL

Chapter 48: Slowly Changing Dimensions

Learning Objectives

Implement all SCD types (0 through 6)

Design SCD Type 2 tables with effective/expiry dates

Use MERGE for automated SCD processing

SCD Type	Strategy	Use Case
Type 0	Never change (fixed)	Country codes, currency codes
Type 1	Overwrite old value	Typo corrections, no history needed
Type 2	Add new row with version history	Customer address, product price
Type 3	Add previous value column	Track only one previous version
Type 4	Separate history table	Active vs historical tables
Type 6	Combine Type 1+2+3 (hybrid)	Comprehensive customer dim

```
-- SCD Type 2 MERGE Implementation
MERGE DimCustomer AS target
USING StagingCustomer AS source
    ON target.CustomerID = source.CustomerID AND target.IsCurrent = 1

WHEN MATCHED AND (
    HASHBYTES('SHA2_256', CONCAT(source.CustomerName,'|',source.Email))
    <> target.RowHash
) THEN
    UPDATE SET target.IsCurrent = 0, target.ExpiryDate = CAST(GETDATE() AS DATE)

WHEN NOT MATCHED BY TARGET THEN
    INSERT (CustomerID, CustomerName, Email, EffectiveDate, ExpiryDate, IsCurrent)
    VALUES (source.CustomerID, source.CustomerName, source.Email,
            CAST(GETDATE() AS DATE), '9999-12-31', 1);
```

Chapter 51: Change Data Capture (CDC)

```
-- Enable CDC
EXEC sys.sp_cdc_enable_db;
EXEC sys.sp_cdc_enable_table
    @source_schema = N'dbo',
    @source_name   = N'Customers',
    @role_name     = NULL;

-- Query CDC changes
DECLARE @from_lsn BINARY(10), @to_lsn BINARY(10);
SET @from_lsn = sys.fn_cdc_get_min_lsn('dbo_Customers');
SET @to_lsn   = sys.fn_cdc_get_max_lsn();
```

```
SELECT
    __$operation,  -- 1=Delete 2=Insert 3=Before Update 4=After Update
    CustomerID, CustomerName, Email
FROM cdc.fn_cdc_get_all_changes_dbo_Customers(@from_lsn, @to_lsn, 'all');
```

Chapter 50: Incremental Load Strategy

```
-- Watermark-based incremental load procedure
CREATE OR ALTER PROCEDURE dbo.usp_IncrementalLoadSales AS BEGIN
    DECLARE @LastLoad DATETIME2, @ThisLoad DATETIME2 = GETDATE();
    SELECT @LastLoad = LastLoadValue FROM ETL_WatermarkControl
    WHERE TableName = 'FactSales';

    BEGIN TRY
        BEGIN TRANSACTION;
            INSERT INTO FactSales (DateKey, CustomerKey, SalesAmount)
            SELECT FORMAT(s.SaleDate,'yyyyMMdd'), dc.CustomerKey, s.Amount
            FROM SourceSales s
            JOIN DimCustomer dc ON s.CustID = dc.CustomerID AND dc.IsCurrent=1
            WHERE s.ModifiedDate > @LastLoad AND s.ModifiedDate <= @ThisLoad;

            UPDATE ETL_WatermarkControl
            SET LastLoadValue = @ThisLoad WHERE TableName = 'FactSales';
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK;
        THROW;
    END CATCH;
END;
```

PART 11: PARTITIONING

Sliding Window Pattern

```
-- Step 1: Create partition function
CREATE PARTITION FUNCTION PF_Sales_Monthly (INT)
AS RANGE RIGHT FOR VALUES (20240101, 20240201, 20240301, 20240401);

-- Step 2: Create partition scheme
CREATE PARTITION SCHEME PS_Sales_Monthly
AS PARTITION PF_Sales_Monthly ALL TO ([PRIMARY]);

-- Step 3: Create partitioned table
CREATE TABLE FactSalesPartitioned (
    SalesKey    BIGINT NOT NULL,
    DateKey     INT NOT NULL,
    SalesAmount DECIMAL(12,2)
) ON PS_Sales_Monthly(DateKey);

-- Switch partition (metadata-only, instant)
ALTER TABLE FactSales SWITCH PARTITION 1 TO FactSales_Archive PARTITION 1;

-- Add new boundary
ALTER PARTITION FUNCTION PF_Sales_Monthly() SPLIT RANGE (20250101);
```

PART 12: BIG DATA & AZURE SQL

OPENROWSET and Azure Synapse

```
-- Query Parquet in ADLS from Synapse Serverless
SELECT CustomerID, SUM(SalesAmount) AS TotalSales
FROM OPENROWSET(
    BULK 'https://storageacct.dfs.core.windows.net/silver/sales/**',
    FORMAT = 'PARQUET'
) WITH (CustomerID INT, SalesAmount DECIMAL(12,2)) AS r
WHERE r.SaleYear = 2024
GROUP BY CustomerID;

-- CTAS: Create Table As Select (Synapse Dedicated Pool)
CREATE TABLE Gold_SalesSummary
WITH (
    DISTRIBUTION = HASH(CustomerKey),
    CLUSTERED COLUMNSTORE INDEX
) AS
SELECT CustomerKey, YEAR(SaleDate) AS SaleYear, SUM(SalesAmount) AS TotalSales
FROM Silver_Sales
GROUP BY CustomerKey, YEAR(SaleDate);
```

Azure SQL Service	Type	Best For
Azure SQL Database	PaaS single database	OLTP apps, SaaS multi-tenant
Azure SQL Managed Instance	PaaS near 100% SQL Server	Lift & shift from on-prem
Synapse Dedicated SQL Pool	MPP DW	Large-scale analytics, DW
Synapse Serverless SQL Pool	Serverless query	Ad-hoc queries on ADLS

Azure SQL Service	Type	Best For
Azure Synapse Spark Pool	Distributed compute	ELT at PB scale

PART 13: SQL INTERVIEW MASTERCLASS -- 200 Q&A

SQL Fundamentals

1. **What is the logical order of SQL clause execution?**

-> FROM/JOIN -> WHERE -> GROUP BY -> HAVING -> SELECT -> DISTINCT -> ORDER BY -> TOP/OFFSET-FETCH. Understanding this prevents confusion about using aliases in WHERE vs HAVING.

2. **What is the difference between DELETE, TRUNCATE, and DROP?**

-> DELETE: DML, logged row by row, supports WHERE, fires triggers, can rollback. TRUNCATE: DDL, minimal logging, resets IDENTITY, no WHERE, no triggers. DROP: removes the entire table structure.

3. **What is a correlated subquery?**

-> A subquery that references columns from the outer query. It executes once per outer row. Often replaceable with window functions for better performance.

4. **What is a CTE and how does it differ from a subquery?**

-> A CTE (Common Table Expression) is a named temporary result set defined in a WITH clause. It improves readability, can be referenced multiple times, and supports recursion. A subquery is inline and can only be used once.

5. **What is a Recursive CTE?**

-> A CTE that references itself. Uses ANCHOR MEMBER (base case) + UNION ALL + RECURSIVE MEMBER. Used for hierarchy traversal (org charts, bill-of-materials, category trees).

6. **Explain the difference between UNION and UNION ALL.**

-> UNION removes duplicate rows (sorts and deduplicates -- costly). UNION ALL keeps all rows including duplicates (faster). In ETL, always prefer UNION ALL unless deduplication is required.

Joins

7. **What is the difference between INNER JOIN and LEFT JOIN?**

-> INNER JOIN returns only rows with matching records in both tables. LEFT JOIN returns all rows from the left table plus matching rows from the right (NULLs for non-matching right columns).

8. **What is a CROSS JOIN?**

-> Returns the Cartesian product -- every combination of rows from both tables. Used for generating combinations, test data, and date/product grids.

9. **What is the difference between CROSS APPLY and OUTER APPLY?**

-> CROSS APPLY is like INNER JOIN with a TVF -- excludes rows where the TVF returns no results. OUTER APPLY is like LEFT JOIN -- includes outer rows even when TVF returns nothing.

Indexes & Performance

10. **What is a covering index?**

-> An index that contains all columns needed by a query, eliminating the need for a Key Lookup back to the clustered index. Achieved by adding INCLUDE columns to a non-clustered index.

11. **What is index fragmentation?**

-> Fragmentation occurs when index page order does not match physical disk order. Fix: REORGANIZE < 30%, REBUILD > 30%.

12. **What is parameter sniffing?**

-> SQL Server compiles and caches execution plans based on the first parameter values seen. Solutions: OPTION(RECOMPILE), OPTION(OPTIMIZE FOR UNKNOWN), local variable trick.

13. **What is a Columnstore index?**

-> Data is stored by column rather than by row. Achieves 5-10x compression and 100x faster aggregation queries. Best for analytics/DW workloads.

Window Functions

14. **What is the difference between ROW_NUMBER(), RANK(), and DENSE_RANK()?**

-> ROW_NUMBER: unique sequential integers, no ties. RANK: ties get same rank, gaps in sequence (1,1,3). DENSE_RANK: ties get same rank, no gaps (1,1,2). Use ROW_NUMBER for deduplication, DENSE_RANK for top-N-per-group.

15. **How do you calculate a running total using a window function?**

-> SUM(Amount) OVER (ORDER BY Date ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW). The ROWS frame is important -- without it, RANGE is used which includes all tied dates.

Transactions & Locking

16. **What are ACID properties?**

-> Atomicity: all-or-nothing. Consistency: valid state before and after. Isolation: concurrent transactions don't interfere. Durability: committed data survives failures.

17. **What is a deadlock?**

-> A deadlock occurs when two sessions each hold a resource the other needs. SQL Server detects deadlocks automatically and kills the session with lowest DEADLOCK_PRIORITY.

18. **What is snapshot isolation?**

-> Snapshot isolation uses row versioning in tempdb to provide readers a consistent view without blocking writers. Eliminates reader-writer deadlocks.

Data Warehousing

19. **What is the difference between Star Schema and Snowflake Schema?**

-> Star Schema: fact table surrounded by denormalized dimension tables. Simple, fewer joins, better BI tool performance. Snowflake: dimension tables are normalized into sub-dimensions. More storage efficient but more joins.

20. **What is SCD Type 2?**

-> Tracks historical changes by adding a new row for each change. Uses EffectiveDate, ExpiryDate, and IsCurrent columns. Allows time-travel queries.

21. **What is a surrogate key?**

-> A system-generated integer key (identity column) used as the primary key in dimension tables. Independent of business logic, stable across source system changes.

22. **What is CDC (Change Data Capture)?**

-> CDC reads the SQL Server transaction log and captures row-level INSERT/UPDATE/DELETE changes. Enables efficient incremental data extraction.

Azure & Synapse

23. **What is CTAS in Azure Synapse Analytics?**

-> Create Table As Select -- the primary method for creating permanent tables in Synapse. Supports specifying DISTRIBUTION and INDEX options. Much faster than INSERT...SELECT for large datasets.

24. **What are the distribution options in Synapse Dedicated SQL Pool?**

-> HASH: distributes rows by hash of a column -- best for large tables. ROUND_ROBIN: even distribution for staging tables. REPLICATE: copies entire table to all distributions for small dimension tables.

25. **What is OPENROWSET in Synapse Serverless?**

-> A T-SQL function that reads files directly from Azure Data Lake Storage. Supports CSV, Parquet, Delta Lake formats. Ideal for ad-hoc exploration.

PART 14: 100 SQL CODING INTERVIEW PROBLEMS

Ranking & Salary Problems

Problem 1: Nth Highest Salary

Problem: Find the Nth highest salary from the Employees table.

```
CREATE FUNCTION dbo.GetNthHighestSalary(@N INT)
RETURNS TABLE AS RETURN (
    WITH Ranked AS (
        SELECT Salary, DENSE_RANK() OVER (ORDER BY Salary DESC) AS dr
        FROM Employees
    )
    SELECT DISTINCT Salary FROM Ranked WHERE dr = @N
);
SELECT * FROM dbo.GetNthHighestSalary(3);
```

Tip: DENSE_RANK handles ties -- if two employees share salary rank 2, OFFSET-FETCH may miss them.

Problem 2: Top N Salaries Per Department

Problem: Find employees with the top 3 salaries in each department.

```
WITH Ranked AS (
    SELECT EmpID, Name, DeptID, Salary,
           DENSE_RANK() OVER (PARTITION BY DeptID ORDER BY Salary DESC) AS dr
    FROM Employees
)
SELECT EmpID, Name, DeptID, Salary
FROM Ranked WHERE dr <= 3
ORDER BY DeptID, Salary DESC;
```

Problem 3: Find Duplicate Records

Problem: Find all duplicate email addresses in the Customers table.

```
-- Find duplicates
SELECT Email, COUNT(*) AS DuplicateCount
FROM Customers GROUP BY Email HAVING COUNT(*) > 1;

-- Delete duplicates, keep lowest CustomerID
WITH Dupes AS (
    SELECT CustomerID,
           ROW_NUMBER() OVER (PARTITION BY Email ORDER BY CustomerID) AS rn
    FROM Customers
)
DELETE FROM Dupes WHERE rn > 1;
```

Problem 4: Employees Earning More Than Their Manager

Problem: List employees who earn more than their direct manager.

```
SELECT e.Name AS Employee, e.Salary AS EmpSalary,
       m.Name AS Manager, m.Salary AS MgrSalary
FROM Employees e
JOIN Employees m ON e.ManagerID = m.EmpID
WHERE e.Salary > m.Salary;
```

Date & Time Problems

Problem 5: Consecutive Login Days

Problem: Find users who logged in for 3 or more consecutive days.

```
WITH LoginDates AS (
    SELECT UserID, LoginDate,
           ROW_NUMBER() OVER (PARTITION BY UserID ORDER BY LoginDate) AS rn
    FROM Logins
),
DateGroups AS (
    SELECT UserID, LoginDate,
           DATEADD(DAY, -rn, LoginDate) AS GroupDate
    FROM LoginDates
)
SELECT UserID, MIN(LoginDate) AS StartDate, MAX(LoginDate) AS EndDate,
       COUNT(*) AS ConsecutiveDays
FROM DateGroups GROUP BY UserID, GroupDate
HAVING COUNT(*) >= 3
ORDER BY UserID, StartDate;
```

Tip: Island detection trick: subtract row_number from date. Consecutive dates get the same GroupDate.

Problem 6: Gap and Island Problem

Problem: Find continuous date ranges where a product was in stock.

```
WITH Numbered AS (
    SELECT ProductID, StockDate,
           ROW_NUMBER() OVER (PARTITION BY ProductID ORDER BY StockDate) AS rn
    FROM StockDates
),
Groups AS (
    SELECT ProductID, StockDate,
           DATEADD(DAY, -rn, StockDate) AS GroupDate FROM Numbered
)
SELECT ProductID, MIN(StockDate) AS IslandStart, MAX(StockDate) AS IslandEnd
FROM Groups GROUP BY ProductID, GroupDate
ORDER BY ProductID, IslandStart;
```

Problem 7: Median Salary Calculation

Problem: Calculate the median salary.

```
WITH Sorted AS (
    SELECT Salary,
           ROW_NUMBER() OVER (ORDER BY Salary) AS rn,
           COUNT(*) OVER () AS total
    FROM Employees
)
SELECT AVG(CAST(Salary AS DECIMAL(10,2))) AS Median
FROM Sorted WHERE rn IN ((total+1)/2, (total+2)/2);

-- Or with PERCENTILE_CONT:
SELECT DISTINCT PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY Salary)
    OVER () AS Median FROM Employees;
```

Running Totals & Aggregations

Problem 8: Running Total of Sales

Problem: Calculate cumulative sales ordered by date.

```
SELECT SaleDate, Amount,
    SUM(Amount) OVER (
        ORDER BY SaleDate
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
```

```
    ) AS RunningTotal
FROM DailySales ORDER BY SaleDate;
```

Problem 9: Month-Over-Month Growth Rate

Problem: Calculate month-over-month revenue growth percentage.

```
WITH Monthly AS (
    SELECT FORMAT(SaleDate,'yyyy-MM') AS Month, SUM(Amount) AS Revenue
    FROM Sales GROUP BY FORMAT(SaleDate,'yyyy-MM')
)
SELECT Month, Revenue,
    ROUND(100.0 * (Revenue - LAG(Revenue) OVER (ORDER BY Month))
          / NULLIF(LAG(Revenue) OVER (ORDER BY Month), 0), 2) AS GrowthPct
FROM Monthly ORDER BY Month;
```

Problem 10: 7-Day Moving Average

Problem: Calculate 7-day rolling average of daily sales.

```
SELECT SaleDate, Amount,
    AVG(Amount) OVER (
        ORDER BY SaleDate ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
    ) AS MovAvg7Day
FROM DailySales ORDER BY SaleDate;
```

Problem 11: Pivot Monthly Sales by Year

Problem: Show total sales for each month as columns in a single row per year.

```
SELECT SaleYear, [Jan],[Feb],[Mar],[Apr],[May],[Jun],
                 [Jul],[Aug],[Sep],[Oct],[Nov],[Dec]
FROM (
    SELECT YEAR(SaleDate) AS SaleYear,
           DATENAME(MONTH,SaleDate) AS MonthName, Amount FROM Sales
) src
PIVOT (
    SUM(Amount) FOR MonthName IN
    ([Jan],[Feb],[Mar],[Apr],[May],[Jun],[Jul],[Aug],[Sep],[Oct],[Nov],[Dec])
) pvt ORDER BY SaleYear;
```

Hierarchy & Recursive Problems

Problem 12: Org Chart Hierarchy

Problem: Display the complete reporting hierarchy from CEO downward.

```
WITH Hierarchy AS (
    SELECT EmpID, Name, ManagerID, 0 AS Level
    FROM Employees WHERE ManagerID IS NULL
    UNION ALL
    SELECT e.EmpID, e.Name, e.ManagerID, h.Level + 1
    FROM Employees e JOIN Hierarchy h ON e.ManagerID = h.EmpID
)
SELECT REPLICATE('  ', Level) + Name AS IndentedName, Level
FROM Hierarchy ORDER BY Level, Name;
```

Data Warehouse & ETL Problems

Problem 13: Find Latest Record Per Group (SCD lookup)

Problem: For each customer, get their current active dimension row.

```
SELECT CustomerKey, CustomerID, CustomerName, City
FROM DimCustomer WHERE IsCurrent = 1;

-- Without IsCurrent flag:
WITH Latest AS (
    SELECT *, ROW_NUMBER() OVER
        (PARTITION BY CustomerID ORDER BY EffectiveDate DESC) AS rn
    FROM DimCustomer
)
SELECT CustomerKey, CustomerID, CustomerName FROM Latest WHERE rn = 1;
```

Problem 14: Calculate Customer Retention Rate

Problem: Find what % of customers who bought in month N also bought in month N+1.

```
WITH MonthlyBuyers AS (
    SELECT CustomerID,
           DATEFROMPARTS(YEAR(OrderDate),MONTH(OrderDate),1) AS Month
    FROM Orders GROUP BY CustomerID,
    DATEFROMPARTS(YEAR(OrderDate),MONTH(OrderDate),1)
)
SELECT a.Month, COUNT(DISTINCT a.CustomerID) AS Buyers,
    ROUND(100.0 * COUNT(DISTINCT b.CustomerID)
    / NULLIF(COUNT(DISTINCT a.CustomerID),0), 2) AS RetentionPct
FROM MonthlyBuyers a
LEFT JOIN MonthlyBuyers b ON a.CustomerID = b.CustomerID
    AND b.Month = DATEADD(MONTH, 1, a.Month)
GROUP BY a.Month ORDER BY a.Month;
```

Problem 15: Cumulative Distribution of Sales

Problem: Classify customers into quartiles based on total spending.

```
WITH CustomerSpend AS (
    SELECT CustomerID, SUM(TotalAmount) AS TotalSpend
    FROM Orders GROUP BY CustomerID
),
Ranked AS (
    SELECT CustomerID, TotalSpend,
           NTILE(4) OVER (ORDER BY TotalSpend DESC) AS Quartile
    FROM CustomerSpend
)
SELECT CustomerID, TotalSpend,
    CASE Quartile
        WHEN 1 THEN 'Top 25% - Platinum'
        WHEN 2 THEN 'Top 50% - Gold'
        WHEN 3 THEN 'Top 75% - Silver'
        ELSE 'Bottom 25% - Bronze'
    END AS CustomerTier
FROM Ranked ORDER BY TotalSpend DESC;
```

BEST PRACTICES & QUICK REFERENCE

SQL Server Performance Tuning Checklist

Area	Check	Action
Indexes	Missing index warnings in execution plan	Create covering index with INCLUDE columns
Indexes	High fragmentation > 30%	REBUILD index; schedule regular maintenance
Query	Implicit data type conversion	Match parameter types exactly to column types
Query	Function on indexed column in WHERE	Rewrite as sargable: date range instead of YEAR()
Query	SELECT *	List only needed columns; enables index covering
Query	NOT IN with nullable subquery	Replace with NOT EXISTS
Stats	Outdated statistics	UPDATE STATISTICS WITH FULLSCAN on key tables
Plan	Parameter sniffing	Add OPTION(RECOMPILE) or use OPTIMIZE FOR UNKNOWN
Blocking	Long-running transactions	Keep transactions short; commit ASAP
Concurrency	Reader-writer blocking	Enable READ_COMMITTED_SNAPSHOT isolation

Data Warehouse Design Checklist

- Define grain before choosing facts -- every fact must be at the stated grain
- Use surrogate keys (INT IDENTITY) for all dimension tables
- Always create a DimDate populated for at least 10 years into the future
- Store row hash (HASHBYTES) on dimension rows for fast change detection
- Partition large fact tables by date (DateKey INT YYYYMMDD)
- Use Clustered Columnstore Index on all large fact tables in Synapse
- Log every ETL run: start time, end time, rows loaded, status, errors
- Implement SCD Type 2 for any dimension that may change over time
- Test incremental loads for idempotency -- running twice produces same result

Senior SQL Engineer Summary

Mastering SQL is about understanding WHY the engine does what it does, not just memorizing syntax.

Every performance problem is either missing statistics, wrong index strategy, bad query structure, or resource pressure.

Data Warehouse design quality directly determines BI tool performance and analyst productivity.

Azure Synapse requires different patterns than SQL Server -- always specify DISTRIBUTION and INDEX.

The best SQL is readable, maintainable, and performant -- optimize in that order.

Window functions + CTEs + MERGE cover 80% of advanced SQL needs -- master these first.